

Cours Python — Jour 4 (partie 2) : les types de données complexes

1 Cours Python — Jour 4 (partie 2) : les types de données complexes

1.1 Introduction

Cette séance est la **deuxième partie du Jour 4**. La première partie était consacrée aux chaînes de caractères et à une première découverte des structures conditionnelles. Nous allons maintenant nous concentrer **exclusivement** sur les types de données un peu plus complexes que propose Python pour organiser et manipuler des ensembles de valeurs :

- les **listes** (`list`),
- les **tuples** (`tuple`),
- les **ensembles** (`set`),
- les **dictionnaires** (`dict`),
- les **ranges** (`range`),

ainsi que les **méthodes de base** associées à chacun de ces types.

Jusqu'ici, nos variables ne contenaient qu'**une seule valeur** à la fois (un nombre, une chaîne, un booléen). Ces cinq types permettent au contraire de regrouper **plusieurs valeurs** dans une seule variable : c'est la base de toute manipulation de données un peu sérieuse. Un administrateur réseau ne travaille jamais avec une seule adresse IP ou un seul port isolé : il manipule des **parcs d'équipements**, des **tables de routage**, des **ensembles de ports autorisés**, des **dictionnaires de configuration**. Ce sont exactement les types que nous allons étudier aujourd'hui.

Objectifs du jour :

- savoir créer, parcourir et modifier une liste, y compris des listes imbriquées,
- comprendre l'immuabilité des tuples et savoir les déballer (*unpacking*),
- utiliser les ensembles pour éliminer des doublons et effectuer des opérations ensemblistes (union, intersection, différence),
- créer, consulter et mettre à jour un dictionnaire, y compris imbriqué,
- utiliser `range()` pour générer des suites de nombres de façon efficace,

- savoir choisir le **type le plus adapté** à un problème donné.

Notion transversale : la mutabilité. On dit qu'un objet est **mutable** lorsqu'on peut modifier son contenu après sa création (la liste et le dictionnaire, par exemple). On dit qu'il est **immuable** (*immutable*) lorsque c'est impossible : toute « modification » crée en réalité un nouvel objet (le tuple, comme la chaîne de caractères déjà vue au Jour 4). Cette distinction revient sans cesse dans ce cours : elle détermine ce que l'on peut faire avec chaque type, et lequel choisir selon la situation.

1.2 Les listes (list)

1.2.1 Qu'est-ce qu'une liste ?

Une liste est une **séquence ordonnée et modifiable** d'éléments. Elle peut contenir des éléments de n'importe quel type, et même mélanger plusieurs types différents dans la même liste.

Caractéristiques :

- **Ordonnée** : l'ordre des éléments est conservé tel qu'il a été saisi,
- **Indexée** : chaque élément possède un indice, qui commence à 0,
- **Modifiable** (*mutable*) : on peut ajouter, supprimer ou modifier des éléments après création,
- **Hétérogène** : elle peut mélanger `int`, `str`, `bool`, une autre liste...
- **Doublons autorisés** : une même valeur peut apparaître plusieurs fois.

Notation : des crochets `[ ]`.

1.2.2 Création et accès (indexation, slicing)

```
ports_tcp = [21, 22, 23, 25, 53, 80, 110, 143, 443, 3306]
serveurs   = ["web-01", "web-02", "db-01", "proxy"]
mixte      = [True, "SSH", 22, 3.14, None]
vide       = []

# Indexation positive : de gauche à droite, commence à 0
print(ports_tcp[0])    # 21 (premier element)
print(ports_tcp[1])    # 22 (deuxieme element)

# Indexation negative : de droite à gauche, commence à -1
print(ports_tcp[-1])   # 3306 (dernier element)
print(ports_tcp[-2])   # 443 (avant-dernier)

# Slicing [debut:fin:pas] -- la borne "fin" est TOUJOURS EXCLUE
print(ports_tcp[2:5])  # [23, 25, 53]          indices 2, 3, 4
print(ports_tcp[:4])   # [21, 22, 23, 25]      les 4 premiers
print(ports_tcp[6:])   # [110, 143, 443, 3306] du 7e à la fin
print(ports_tcp[::2])  # un element sur deux
print(ports_tcp[::-1]) # liste inversee
```

```
print(len(ports_tcp))      # 10 : nombre d'elements
print(80 in ports_tcp)     # True : test d'appartenance
print(8080 in ports_tcp)   # False
```

Piège classique du slicing : `ports_tcp[2:5]` renvoie les éléments aux indices 2, 3 et 4 — **jamais** l'élément d'indice 5. La borne de fin d'un slice est toujours exclue, exactement comme pour les chaînes de caractères vues au Jour 4.

1.2.3 Modification d'une liste

Contrairement au tuple ou à la chaîne, la liste est **mutable** : on peut la modifier directement, sans créer de nouvel objet.

```
equipements = ["router", "switch", "firewall"]

# Modifier UN element par son indice
equipements[1] = "switch-L3"
print(equipements)  # ['router', 'switch-L3', 'firewall']

# Modifier PLUSIEURS elements avec un slice
equipements[1:3] = ["switch-L3", "firewall-ng", "IDS"]
print(equipements)  # ['router', 'switch-L3', 'firewall-ng', 'IDS']

# Supprimer un element par SA VALEUR
equipements.remove("IDS")
print(equipements)  # ['router', 'switch-L3', 'firewall-ng']

# Supprimer un element par SON INDICE, avec l'instruction del
del equipements[0]
print(equipements)  # ['switch-L3', 'firewall-ng']
```

1.2.4 Méthodes de base des listes

Méthode	Effet
<code>.append(x)</code>	ajoute <code>x</code> à la fin de la liste
<code>.insert(i, x)</code>	insère <code>x</code> à l'indice <code>i</code> (décale les éléments suivants)
<code>.extend(iter)</code>	ajoute tous les éléments d'un itérable à la fin
<code>.remove(x)</code>	supprime la première occurrence de <code>x</code> (erreur si absent)
<code>.pop(i)</code>	supprime et retourne l'élément d'indice <code>i</code> (par défaut, le dernier)
<code>.index(x)</code>	renvoie l'indice de la première occurrence de <code>x</code>
<code>.count(x)</code>	compte le nombre d'occurrences de <code>x</code>
<code>.sort()</code>	trie la liste sur place (modifie la liste d'origine)
<code>.reverse()</code>	inverse l'ordre des éléments sur place
<code>.copy()</code>	renvoie une copie indépendante (superficielle) de la liste
<code>.clear()</code>	vide la liste (elle continue d'exister, mais devient vide)

```

acl = ["deny", "allow", "deny", "allow", "allow"]

acl.append("log")           # ajoute a la fin
acl.insert(0, "default-deny") # insere en tete
acl.extend(["nat", "forward"]) # ajoute plusieurs elements a la fin
acl.remove("deny")          # retire la 1ere occurrence de "deny"

dernier = acl.pop()          # retire ET renvoie le dernier element
deuxieme = acl.pop(1)        # retire ET renvoie l'element d'indice 1

print(acl.index("allow"))    # indice de la 1ere occurrence de "allow"
print(acl.count("allow"))    # nombre d'occurrences de "allow"

notes = [14, 8, 17, 5, 12, 9, 11]
notes.sort()                 # tri croissant, SUR PLACE
print(notes)                 # [5, 8, 9, 11, 12, 14, 17]
notes.sort(reverse=True)     # tri decroissant
print(notes)                 # [17, 14, 12, 11, 9, 8, 5]

```

Attention, piège très fréquent : `.sort()` trie la liste **sur place** et renvoie `None`. Écrire `notes = notes.sort()` est une erreur classique : la variable `notes` devient alors `None` et le contenu de la liste est perdu ! Si l'on souhaite **conserver la liste d'origine** et obtenir une **nouvelle** liste triée, il faut utiliser la fonction `sorted()` (et non une méthode) :

```

ips = ["10.0.0.1", "192.168.1.1", "172.16.0.1"]
ips_triees = sorted(ips) # nouvelle liste, ips n'est PAS modifie
print(ips_triees)
print(ips)               # toujours dans l'ordre d'origine

```

```

# .copy() : copie independante (modifier la copie ne touche pas l'originale)
original = [1, 2, 3]
copie = original.copy()
copie.append(99)
print(original) # [1, 2, 3]      inchangee
print(copie)    # [1, 2, 3, 99]

# .clear() : vide la liste
temp = [1, 2, 3]
temp.clear()
print(temp) # []

```

Copie superficielle : `.copy()` duplique la liste elle-même, mais pas les objets mutables qu'elle contient. Si la liste contient d'autres listes ou des dictionnaires, ceux-ci restent **partagés** entre l'original et la copie. Pour l'instant, avec des listes de valeurs simples (nombres, chaînes), cela ne pose pas de problème.

1.2.5 Listes imbriquées (matrices)

Une liste peut contenir d'autres listes : on parle de **liste imbriquée**, souvent utilisée pour représenter un tableau à deux dimensions (une matrice), par exemple un petit tableau de bord réseau.

```
# Tableau de bord réseau : [hote, ip, statut, latence_ms]
parc_reseau = [
    ["router-01", "192.168.1.1", "UP", 2],
    ["switch-01", "192.168.1.2", "UP", 1],
    ["srv-web", "192.168.1.10", "UP", 8],
    ["srv-db", "192.168.1.11", "DOWN", 0],
    ["firewall", "10.0.0.1", "UP", 5],
]

print(parc_reseau[0][0]) # 'router-01' ligne 0, colonne 0
print(parc_reseau[2][1]) # '192.168.1.10' ligne 2, colonne 1

print("Equipements DOWN :")
for equipement in parc_reseau:
    if equipement[2] == "DOWN":
        print(f"ALERTE : {equipement[0]} ({equipement[1]}) est hors ligne !")

# Compréhension de liste pour extraire une colonne (les latences des UP)
latences_up = [e[3] for e in parc_reseau if e[2] == "UP"]
print(latences_up) # [2, 1, 8, 5]
print(sum(latences_up) / len(latences_up)) # latence moyenne
```

Pour accéder à un élément d'une liste imbriquée, on enchaîne les crochets : le premier indice sélectionne la **ligne** (l'équipement), le second sélectionne la **colonne** (l'information sur cet équipement).

1.3 Les tuples (tuple)

1.3.1 Qu'est-ce qu'un tuple ?

Un tuple est une **séquence ordonnée et immuable** d'éléments. Contrairement à la liste, on ne peut **pas** le modifier après sa création : ni ajouter, ni supprimer, ni changer un élément.

Caractéristiques :

- **Ordonné** et **indexé**, comme une liste,
- **Immuable** : impossible à modifier après création,
- **Hétérogène** : peut mélanger les types,
- **Hashable** (si ses éléments le sont) : peut donc servir de **clé de dictionnaire** ou d'élément d'un **set**, contrairement à la liste.

Quand utiliser un tuple plutôt qu'une liste ? Un tuple convient à des données **constantes** qui ne doivent pas changer (des coordonnées GPS, une configuration figée, la valeur de retour

d'une fonction). Une liste convient à des données **amenées à évoluer** (une file d'équipements, des résultats accumulés au fil d'un programme).

Notation : des parenthèses (), ou simplement une virgule.

1.3.2 Création et immuabilité

```
coord_serveur = (48.8566, 2.3522)           # latitude, longitude
config_ssh    = ("192.168.1.10", 22, "SSH")
seul_element  = (42,)                       # attention : la virgule est OBLIGATOIRE
sans_parenth  = 10, 20, 30                 # les parentheses sont facultatives
vide_tuple    = ()

print(config_ssh[0])    # '192.168.1.10'
print(config_ssh[1])    # 22
print(config_ssh[1:])   # (22, 'SSH')

# Toute tentative de modification leve une TypeError
try:
    config_ssh[1] = 2222
except TypeError as e:
    print(f"Immutabilite confirmee : {e}")
```

Piège classique : (42) n'est **pas** un tuple, c'est simplement l'entier 42 entre parenthèses. Pour créer un tuple à un seul élément, la virgule finale est **obligatoire** : (42,).

1.3.3 Déballage (unpacking)

Le **déballage** consiste à affecter en une seule ligne chaque élément d'un tuple (ou de toute séquence) à une variable distincte.

```
ip, port, protocole = config_ssh
print(f"IP={ip} Port={port} Protocole={protocole}")

# L'étoile (*) absorbe "le reste" dans une liste
premier, *reste = (10, 20, 30, 40, 50)
print(premier, reste)    # 10 [20, 30, 40, 50]

*debut, dernier = (10, 20, 30, 40, 50)
print(debut, dernier)    # [10, 20, 30, 40] 50

# Cas concret : déballage d'une table de routage (liste de tuples)
routes = [
    ("192.168.1.0/24", "eth0", "0.0.0.0", 110),
    ("10.0.0.0/8", "eth1", "10.0.0.254", 20),
    ("0.0.0.0/0", "eth0", "192.168.1.1", 1),
]
for reseau, interface, passerelle, metrique in routes:
    print(f"{reseau:18} via {passerelle:15} dev {interface} metric {metrique}")
```

```
# Le "swap" (échange) de deux variables utilise un tuple implicite
a, b = 100, 200
a, b = b, a
print(a, b)    # 200 100
```

1.3.4 Méthodes de base des tuples

Un tuple étant immuable, il n'offre que **deux** méthodes : il est impossible d'y ajouter, retirer ou trier des éléments sur place.

Méthode	Effet
<code>.count(x)</code>	compte le nombre d'occurrences de <code>x</code>
<code>.index(x)</code>	renvoie l'indice de la première occurrence de <code>x</code>

```
codes_http = (200, 404, 200, 500, 404, 200, 301, 404)
print(codes_http.count(200))    # 3
print(codes_http.index(500))    # 3 (indice de la première occurrence)

# Conversion tuple <-> liste, pour pouvoir modifier temporairement les données
liste_ports = list(config_ssh)  # tuple -> liste (modifiable)
liste_ports.append(2222)
tuple_ports = tuple(liste_ports) # liste -> tuple (de nouveau immuable)
print(tuple_ports)
```

1.4 Les ensembles (set et frozenset)

1.4.1 Qu'est-ce qu'un set ?

Un **set** est une **collection non ordonnée d'éléments uniques**, directement inspirée de la notion mathématique d'ensemble.

Caractéristiques :

- **Non ordonné** : pas d'indice, pas de slicing possible,
- **Unique** : chaque valeur ne peut apparaître qu'une seule fois,
- **Modifiable** : on peut ajouter ou retirer des éléments,
- les éléments doivent être **hashables** (donc pas de liste ni de dictionnaire à l'intérieur d'un set).

Quand utiliser un set ? Pour supprimer automatiquement des doublons, pour tester une appartenance très rapidement (en moyenne, un test `x in mon_set` est bien plus rapide qu'un `x in`

`ma_liste` lorsque la collection est grande), ou pour effectuer des opérations ensemblistes (union, intersection, différence).

Piège de notation : les accolades `{ }` **vides** créent un **dictionnaire** vide, pas un set
! Pour un set vide, il faut impérativement écrire `set()`.

1.4.2 Création et unicité

```
protocoles = {"TCP", "UDP", "ICMP", "TCP", "UDP", "ARP"} # doublons supprimees
ports_ouverts = {22, 80, 443, 80, 22, 8080} # doublons supprimees
set_vide = set() # PAS {} : ce serait un dict vide

print(protocoles) # ordre non garanti, chaque valeur une seule fois

# Convertir une liste en set pour eliminer les doublons
logs_ips = ["10.0.0.1", "192.168.1.5", "10.0.0.1", "8.8.8.8", "192.168.1.5"]
ips_uniques = set(logs_ips)
print(ips_uniques) # {'10.0.0.1', '192.168.1.5', '8.8.8.8'}

print("TCP" in protocoles) # True (test tres rapide, en moyenne)
print("FTP" in protocoles) # False
```

1.4.3 Méthodes de base et opérateurs ensemblistes

Méthode / opérateur	Effet
<code>.add(x)</code>	ajoute l'élément <code>x</code>
<code>.remove(x)</code>	supprime <code>x</code> (erreur si absent)
<code>.discard(x)</code>	supprime <code>x</code> sans erreur s'il est absent
<code>.pop()</code>	supprime et renvoie un élément arbitraire
<code>.clear()</code>	vide le set
<code>A B</code> ou <code>.union(B)</code>	union : tous les éléments de <code>A</code> et de <code>B</code>
<code>A & B</code> ou <code>.intersection(B)</code>	intersection : éléments présents dans <code>A</code> et <code>B</code>
<code>A - B</code> ou <code>.difference(B)</code>	différence : éléments de <code>A</code> absents de <code>B</code>
<code>A ^ B</code>	différence symétrique : dans <code>A</code> ou <code>B</code> , mais pas dans les deux
<code>.issubset(B)</code> ou <code>A <= B</code>	vrai si <code>A</code> est inclus dans <code>B</code>
<code>.issuperset(B)</code> ou <code>A >= B</code>	vrai si <code>A</code> contient <code>B</code>
<code>.isdisjoint(B)</code>	vrai si <code>A</code> et <code>B</code> n'ont aucun élément commun
<code>.update(B)</code>	ajoute à <code>A</code> tous les éléments de <code>B</code> (sur place)

```
ports_ouverts = {22, 80, 443}
ports_ouverts.add(3306)
ports_ouverts.remove(80)
ports_ouverts.discard(9999) # 9999 absent : aucune erreur (contrairement a remove)

services_web = {"HTTP", "HTTPS", "FTP", "SMTP", "DNS"}
services_securises = {"HTTPS", "SSH", "SFTP", "DNS", "SMTPS"}
```



```

print(services_web | services_securises)    # union
print(services_web & services_securises)    # intersection : {'HTTPS', 'DNS'}
print(services_web - services_securises)    # dans web, pas dans securises
print(services_web ^ services_securises)    # dans l'un OU l'autre, pas les deux

admin_ports = {22, 443}
tous_ports = {22, 80, 443, 8080, 3306}
print(admin_ports.issubset(tous_ports))     # True
print(admin_ports <= tous_ports)           # True (equivalent)

set_a = {1, 2, 3}
set_b = {4, 5, 6}
print(set_a.isdisjoint(set_b))             # True : aucun element commun

```

Application réseau : dans une politique de pare-feu, deux ensembles de ports autorisés (par exemple une zone DMZ et une zone interne) se comparent naturellement avec des opérations ensemblistes : l'intersection donne les ports communs, la différence donne les ports ouverts uniquement d'un côté — une information précieuse pour un audit de sécurité.

1.4.4 Le frozenset

Un **frozenset** est un **set immuable** : on ne peut ni ajouter ni retirer d'éléments après sa création. Comme il est immuable, il est **hashable**, et peut donc servir de clé de dictionnaire ou être placé à l'intérieur d'un autre set — ce qu'un **set** ordinaire ne permet pas.

```

ports_restreints = frozenset({22, 80, 443})
print(443 in ports_restreints)             # True
print(ports_restreints & {80, 8080, 443})  # {80, 443} : intersection possible

# Utilisation comme cle de dictionnaire (impossible avec un set normal)
politiques = {
    frozenset({22, 443}): "admin",
    frozenset({80, 8080}): "web",
}
print(politiques[frozenset({22, 443})])    # 'admin'

```

1.5 Les dictionnaires (dict)

1.5.1 Qu'est-ce qu'un dictionnaire ?

Un dictionnaire est une collection de paires **clé : valeur**. Chaque clé est **unique** et donne accès à sa valeur associée. C'est l'équivalent d'un annuaire ou d'une table de configuration.

Caractéristiques :

- **Ordonné** depuis Python 3.7 : l'ordre d'insertion des clés est conservé,
- **Clés uniques** : une clé ne peut apparaître qu'une seule fois,
- **Modifiable** : on peut ajouter, modifier ou supprimer des paires,
- les **clés** doivent être hashables (`str`, `int`, `tuple`...), les **valeurs** peuvent être de n'importe quel type.

Notation : des accolades avec deux-points, `{"clé": valeur}`.

1.5.2 Création et accès

```
serveur = {
    "hostname": "srv-web-01",
    "ip":       "192.168.1.10",
    "os":       "Ubuntu 22.04",
    "ram_go":   16,
    "actif":    True,
}

print(serveur["hostname"])    # 'srv-web-01'

# Acces securise avec .get() : renvoie None (ou une valeur par default)
# si la cle est absente, AU LIEU de lever une erreur
print(serveur.get("ram_go"))    # 16
print(serveur.get("cpu", "N/A")) # 'N/A' : cle absente, valeur par default

# Modifier / ajouter une valeur
serveur["ram_go"] = 32          # modifie une cle existante
serveur["cpu"]     = "Intel Xeon" # cree une nouvelle cle

# Supprimer une cle
del serveur["actif"]

# Tester l'existence d'une cle
print("ip" in serveur)    # True
print("gpu" in serveur)   # False
```

Pourquoi préférer `.get()` à `serveur["cle"]` ? Accéder à une clé absente avec les crochets (`serveur["gpu"]` par exemple) lève une `KeyError` et arrête le programme si elle n'est pas gérée. `.get()` renvoie simplement `None` (ou la valeur par défaut fournie), ce qui est souvent plus sûr lorsqu'on ne sait pas à l'avance si la clé existe — par exemple en lisant une configuration réseau potentiellement incomplète.

1.5.3 Méthodes de base des dictionnaires

Méthode	Effet
<code>.get(clé, défaut)</code>	renvoie la valeur, ou défaut (ou <code>None</code>) si absente

Méthode	Effet
<code>.keys()</code>	vue sur toutes les clés
<code>.values()</code>	vue sur toutes les valeurs
<code>.items()</code>	vue sur les couples (clé, valeur)
<code>.update(autre_dict)</code>	fusionne <code>autre_dict</code> (écrase les clés communes)
<code>.pop(clé)</code>	supprime la clé et renvoie sa valeur
<code>.popitem()</code>	supprime et renvoie la dernière paire insérée
<code>.setdefault(clé, val)</code>	renvoie la valeur si la clé existe, sinon l'insère avec <code>val</code>
<code>.copy()</code>	copie superficielle du dictionnaire
<code>.clear()</code>	vide le dictionnaire

```
config_reseau = {
    "gateway": "192.168.1.1",
    "dns_primaire": "8.8.8.8",
    "dns_secondaire": "8.8.4.4",
    "vlan": 10,
    "mtu": 1500,
}

for cle, valeur in config_reseau.items():
    print(f"{cle:16} = {valeur}")

# .update() fusionne un autre dictionnaire (ecrase les cles communes)
config_reseau.update({"mtu": 9000, "jumbo_frames": True})

# .pop() retire une cle ET renvoie sa valeur
vlan_retire = config_reseau.pop("vlan")

# .pop() avec une valeur par défaut : jamais d'erreur
valeur = config_reseau.pop("inexistant", "N/A")

# .setdefault() : insere SEULEMENT si la cle est absente
config_reseau.setdefault("protocole", "IPv4") # insere, car absente
config_reseau.setdefault("gateway", "10.0.0.1") # n'ecrase PAS : cle deja presente
```

1.5.4 Dictionnaires imbriqués

Tout comme une liste peut contenir d'autres listes, un dictionnaire peut contenir d'autres dictionnaires : c'est très courant pour représenter un inventaire structuré.

```
inventaire_reseau = {
    "routeurs": {
        "router-01": {"ip": "192.168.1.1", "modele": "Cisco ISR4321"},
    },
    "serveurs": {
        "srv-web": {"ip": "192.168.1.10", "os": "Ubuntu 22.04"},
        "srv-db": {"ip": "192.168.1.11", "os": "Debian 12"},
    }
}
```

```

    },
}

# Acces en profondeur : on enchaîne les crochets
ip_router = inventaire_reseau["routeurs"]["router-01"]["ip"]
print(ip_router)    # '192.168.1.1'

# Parcours imbriqué : une boucle for dans une autre
for categorie, equipements in inventaire_reseau.items():
    print(f"[{categorie.upper()}]")
    for nom, details in equipements.items():
        print(f"    {nom} : {details.get('ip', '?')}")

```

1.6 Les ranges (range)

1.6.1 Qu'est-ce qu'un range ?

`range()` génère une **séquence d'entiers immuable**, sans stocker tous les nombres en mémoire : les valeurs sont produites **à la demande** (on parle de génération « paresseuse », ou *lazy*). C'est pour cette raison que `range(0, 1_000_000)` ne consomme presque aucune mémoire : Python ne conserve que trois nombres (**start**, **stop**, **step**), pas le million d'entiers.

Syntaxe :

Forme	Description	Exemple
<code>range(stop)</code>	de 0 à stop-1	<code>range(5)</code> → 0,1,2,3,4
<code>range(start, stop)</code>	de start à stop-1	<code>range(1, 6)</code> → 1,2,3,4,5
<code>range(start, stop, step)</code>	de start à stop-1, par pas de step	<code>range(0, 256, 16)</code> → 0,16,32...240

1.6.2 Propriétés et conversions

```

r1 = range(10)
r2 = range(0, 256, 16)    # multiples de 16, de 0 à 255
r3 = range(255, -1, -1)   # compte à rebours de 255 à 0 (pas négatif)

print(list(r1))            # [0, 1, 2, ..., 9] -- affichage explicite du contenu
print(r2.start, r2.stop, r2.step)  # 0 256 16
print(len(r2))             # nombre d'éléments, sans les générer tous
print(r2[0], r2[-1])       # premier et dernier élément, comme une liste

# Test d'appartenance : aussi rapide qu'un set, sans générer la séquence
print(128 in r2)           # True
print(100 in r2)           # False

# Conversions
print(list(range(5)))      # [0, 1, 2, 3, 4]

```

```
print(tuple(range(5)))      # (0, 1, 2, 3, 4)
print(set(range(0, 10, 3))) # {0, 3, 6, 9}

# Application : generer les 10 premieres IPs d'un sous-reseau /24
for i in range(1, 11):
    print(f"192.168.0.{i}")
```

Pourquoi ne pas toujours utiliser une liste ? Si l'on doit seulement **parcourir** une suite de nombres (dans un `for`) ou en **tester l'appartenance**, `range()` fait exactement le travail d'une liste de nombres, mais sans en payer le coût mémoire. On ne convertit `range()` en liste (`list(range(...))`) que lorsqu'on a réellement besoin de manipuler tous les éléments comme une liste (les trier, les modifier).

1.7 Synthèse : quel type choisir ?

Type	Ordonné	Modifiable	Doublons	Usage typique
<code>list</code>	Oui	Oui	Oui	données qui évoluent (file d'équipements, résultats)
<code>tuple</code>	Oui	Non	Oui	données constantes (coordonnées, config figée)
<code>set</code>	Non	Oui	Non	doublons à éliminer, tests d'appartenance rapides
<code>dict</code>	Oui (3.7+)	Oui	Non (sur les clés)	associer une valeur à une clé (config, annuaire)
<code>range</code>	Oui	Non	—	générer une suite de nombres sans la stocker

Question à se poser systématiquement : « ai-je besoin de modifier cette collection après sa création ? », « l'ordre compte-t-il ? », « les doublons sont-ils un problème ou une information utile ? ». Les réponses à ces trois questions suffisent, dans la grande majorité des cas, à identifier le type le plus adapté.

1.8 Exercices corrigés

1.8.1 Exercice 1 : gestion d'un parc réseau

Énoncé : gérer un parc d'équipements réseau, chacun représenté par un dictionnaire (`nom`, `ip`, `type`, `statut`, `ping_ms`), avec des fonctions pour ajouter, supprimer, lister les équipements hors ligne, calculer la latence moyenne et produire un rapport.

Concepts utilisés : listes, dictionnaires, compréhensions de liste.

```
def ajouter_equipement(parc, nom, ip, type_eq, statut="UP", ping_ms=0):
    """Ajoute un équipement au parc. Refuse les doublons de nom."""
    noms_existants = {e["nom"] for e in parc} # compréhension de set
    if nom in noms_existants:
        print(f"[ERREUR] '{nom}' existe déjà dans le parc.")
        return
    parc.append({"nom": nom, "ip": ip, "type": type_eq,
```

```

        "statut": statut, "ping_ms": ping_ms})
    print(f"[OK] '{nom}' ajoute.")

def supprimer_equipement(parc, nom):
    """Supprime un équipement par son nom."""
    for i, e in enumerate(parc):
        if e["nom"] == nom:
            parc.pop(i)
            print(f"[OK] '{nom}' supprime.")
            return
    print(f"[ERREUR] '{nom}' introuvable.")

def equipements_down(parc):
    """Renvoie la liste des équipements hors ligne."""
    return [e for e in parc if e["statut"] == "DOWN"]

def latence_moyenne(parc):
    """Latence moyenne des équipements UP uniquement."""
    pings_up = [e["ping_ms"] for e in parc if e["statut"] == "UP" and e["ping_ms"] > 0]
    return sum(pings_up) / len(pings_up) if pings_up else 0

def rapport_parc(parc):
    """Affiche un rapport complet du parc reseau."""
    total = len(parc)
    up = sum(1 for e in parc if e["statut"] == "UP")
    down = total - up

    types = {}
    for e in parc:
        types[e["type"]] = types.get(e["type"], 0) + 1

    print(f"Equipements totaux : {total}    UP : {up}    DOWN : {down}")
    print(f"Latence moyenne      : {latence_moyenne(parc):.1f} ms")
    print(f"Par type                : {types}")
    if down > 0:
        print("Alertes DOWN :")
        for e in equipements_down(parc):
            print(f"  ! {e['nom']} ({e['ip']})")

parc = []
ajouter_equipement(parc, "router-01", "192.168.1.1", "routeur", "UP", 2)
ajouter_equipement(parc, "sw-core", "192.168.1.2", "switch", "UP", 1)
ajouter_equipement(parc, "srv-web", "192.168.1.10", "serveur", "UP", 8)
ajouter_equipement(parc, "srv-db", "192.168.1.11", "serveur", "DOWN", 0)
ajouter_equipement(parc, "router-01", "192.168.1.1", "routeur") # doublon refuse
supprimer_equipement(parc, "sw-core")
rapport_parc(parc)

```

Après ces opérations, le parc contient 3 équipements (router-01, srv-web, srv-db), dont 2 UP et 1 DOWN. La latence moyenne ne prend en compte que les équipements UP avec un ping strictement

positif (router-01=2 et srv-web=8), soit $(2 + 8)/2 = 5.0$ ms.

1.8.2 Exercice 2 : analyse d'un journal réseau

Énoncé : à partir de lignes de log au format "heure niveau ip message", compter les occurrences de chaque niveau, regrouper les messages par adresse IP source, identifier l'IP la plus active et lister les IPs ayant émis au moins une erreur.

Concepts utilisés : dictionnaires, listes, sets, `.get()`.

```
logs_bruts = [
    "08:00:01 INFO 10.0.0.5 connexion SSH etablie",
    "08:01:15 INFO 10.0.0.3 transfert FTP démarre",
    "08:05:30 ERROR 10.0.0.7 tentative bruteforce SSH",
    "08:12:00 INFO 10.0.0.5 fichier transfere",
    "08:15:44 ERROR 10.0.0.7 acces refuse /etc/passwd",
    "08:20:01 WARNING 10.0.0.3 bande passante elevee",
    "08:25:00 INFO 10.0.0.5 deconnexion SSH",
    "08:30:00 ERROR 10.0.0.3 connexion expiree",
    "08:35:00 INFO 10.0.0.9 scan reseau detecte",
    "08:40:00 ERROR 10.0.0.7 flood UDP detecte",
]

# 1) Compter les niveaux avec un dictionnaire "compteur"
niveaux = {}
for ligne in logs_bruts:
    niveau = ligne.split()[1]
    niveaux[niveau] = niveaux.get(niveau, 0) + 1
print("Niveaux :", niveaux)

# 2) Regrouper les messages par IP source
logs_par_ip = {}
for ligne in logs_bruts:
    parties = ligne.split()
    ip = parties[2]
    message = " ".join(parties[3:])
    if ip not in logs_par_ip:
        logs_par_ip[ip] = []
    logs_par_ip[ip].append(message)

for ip, messages in logs_par_ip.items():
    print(f"{ip} : {len(messages)} entree(s)")

# 3) IP la plus active (recherche du maximum sans max())
ip_plus_active, nb_max = None, 0
for ip, messages in logs_par_ip.items():
    if len(messages) > nb_max:
        ip_plus_active, nb_max = ip, len(messages)
print(f"IP la plus active : {ip_plus_active} ({nb_max} entrees)")

# 4) IPs avec au moins une ligne ERROR (un set elimine les doublons)
ips_en_erreur = set()
```

```
for ligne in logs_bruts:
    parties = ligne.split()
    if parties[1] == "ERROR":
        ips_en_erreur.add(parties[2])
print("IPs avec erreur(s) :", ips_en_erreur)
```

Sur ces 10 lignes, on obtient 5 INFO, 4 ERROR et 1 WARNING. Trois IPs (10.0.0.5, 10.0.0.3 et 10.0.0.7) totalisent chacune 3 entrées : le code retient la **première** rencontrée en cas d'égalité, ici 10.0.0.5 — un comportement à connaître lorsqu'on cherche un maximum « à la main ». Les IPs en erreur sont 10.0.0.7 et 10.0.0.3.

1.8.3 Exercice 3 : opérations ensemblistes sur un pare-feu

Énoncé : à partir de deux ensembles de ports autorisés (zone DMZ et zone interne), déterminer les ports communs, les ports propres à chaque zone, l'union de toutes les règles et détecter une éventuelle exposition de ports de base de données en DMZ.

Concepts utilisés : set, opérateurs ensemblistes.

```
ports_dmz      = {80, 443, 8080, 8443, 22, 21, 20}
ports_interne  = {22, 443, 3306, 5432, 8080, 27017, 6379, 5000}

communs        = ports_dmz & ports_interne
dmz_seulement  = ports_dmz - ports_interne
internes_seul  = ports_interne - ports_dmz
toutes_regles  = ports_dmz | ports_interne
asymetrie      = ports_dmz ^ ports_interne

print("Ports communs      :", sorted(communs))
print("Seulement DMZ     :", sorted(dmz_seulement))
print("Seulement internes :", sorted(internes_seul))
print("Union totale      :", sorted(toutes_regles))
print("Asymetrie         :", sorted(asymetrie))

# Analyse de securite : des ports de base de donnees exposes en DMZ ?
ports_bases_de_donnees = {3306, 5432, 27017, 6379}
risque_dmz = ports_dmz & ports_bases_de_donnees
if risque_dmz:
    print("RISQUE : ports DB exposes en DMZ :", risque_dmz)
else:
    print("OK : aucun port de base de donnees en DMZ")
```

Les ports communs aux deux zones sont 22, 443 et 8080. Aucun des ports de bases de données (3306, 5432, 27017, 6379) n'apparaît dans `ports_dmz` : l'intersection `risque_dmz` est donc vide, et le message « OK » s'affiche — exactement le type de vérification automatique qu'un script d'audit de pare-feu peut effectuer.

1.8.4 Exercice 4 : recherche dans une table de routage

Énoncé : à partir d'une table de routage représentée par une liste de tuples (réseau, interface, passerelle, métrique), retrouver la route par défaut et la route ayant la meilleure (plus petite) métrique.

Concepts utilisés : tuple, déballage, parcours de liste.

```
table_routage = [
    ("192.168.1.0/24", "eth0", "0.0.0.0", 110),
    ("10.0.0.0/8", "eth1", "10.0.0.254", 20),
    ("172.16.0.0/16", "eth2", "172.16.0.254", 30),
    ("0.0.0.0/0", "eth0", "192.168.1.1", 1),
]

def trouver_route_defaut(table):
    """Trouve la route par défaut (reseau 0.0.0.0/0)."""
    for reseau, interface, passerelle, metrique in table:
        if reseau == "0.0.0.0/0":
            return reseau, interface, passerelle, metrique
    return None

def route_meilleure_metrrique(table):
    """Renvoie la route ayant la metrique la plus basse."""
    meilleure = table[0]
    for route in table[1:]:
        if route[3] < meilleure[3]:
            meilleure = route
    return meilleure

route_defaut = trouver_route_defaut(table_routage)
if route_defaut:
    reseau, interface, passerelle, metrique = route_defaut
    print(f"Route par défaut : {passerelle} via {interface} (metrique {metrique})")

meilleure = route_meilleure_metrrique(table_routage)
print(f"Meilleure route : {meilleure[0]} via {meilleure[1]} (metrique {meilleure[3]})")
)
```

La route par défaut (0.0.0.0/0) passe par 192.168.1.1 via eth0 avec une métrique de 1 : c'est aussi, dans cet exemple, la route à la meilleure métrique de toute la table.

1.8.5 Exercice 5 : génération d'une plage d'adresses IP

Énoncé : générer automatiquement toutes les adresses IP d'un sous-réseau entre deux bornes d'hôte, puis vérifier efficacement si un hôte donné appartient à cette plage.

Concepts utilisés : range, compréhension de liste.

```
def plage_ips(sous_reseau, debut, fin):
    """Genere la liste des IPs d'un sous-reseau entre deux bornes incluses."""
    prefixe = ".".join(sous_reseau.split(".")[:3]) # ex: "192.168.0"
```

```

    return [f"{prefixe}.{i}" for i in range(debut, fin + 1)]

adresses = plage_ips("192.168.0.0", 10, 20)
print(f"{len(adresses)} adresses generees :", adresses)

# Test d'appartenance efficace avec un objet range (pas besoin de generer la liste)
hotes_valides = range(10, 21)
for octet in [5, 10, 20, 25]:
    resultat = "valide" if octet in hotes_valides else "hors plage"
    print(f"Hote .{octet} : {resultat}")

```

`plage_ips("192.168.0.0", 10, 20)` génère 11 adresses, de 192.168.0.10 à 192.168.0.20 inclus (la borne `fin` doit être incrémentée de 1 puisque `range()` exclut sa borne supérieure). Le test avec `hotes_valides` montre qu'il n'est même pas nécessaire de construire la liste des adresses pour vérifier l'appartenance d'un hôte : l'objet `range` suffit.

1.9 Résumé et récapitulatif

Type	Méthodes clés
<code>list</code>	<code>append</code> , <code>insert</code> , <code>extend</code> , <code>remove</code> , <code>pop</code> , <code>index</code> , <code>count</code> , <code>sort</code> , <code>reverse</code> , <code>copy</code> , <code>clear</code>
<code>tuple</code>	<code>count</code> , <code>index</code> (seulement 2, car immuable)
<code>set</code>	<code>add</code> , <code>remove</code> , <code>discard</code> , <code>pop</code> , <code>clear</code> , <code>union</code> , <code>intersection</code> , <code>difference</code> , <code>update</code>
<code>dict</code>	<code>get</code> , <code>keys</code> , <code>values</code> , <code>items</code> , <code>update</code> , <code>pop</code> , <code>popitem</code> , <code>setdefault</code> , <code>copy</code> , <code>clear</code>
<code>range</code>	propriétés <code>start</code> , <code>stop</code> , <code>step</code> ; conversion via <code>list()</code> , <code>tuple()</code> , <code>set()</code>

- `list [...]` : séquence ordonnée, modifiable, doublons autorisés.
- `tuple (...)` : séquence ordonnée, **immuable**, hashable.
- `set {...}` : collection non ordonnée, éléments **uniques**, modifiable.
- `frozenset` : la version **immuable** et hashable du `set`.
- `dict {clé: valeur}` : paires clé/valeur, clés uniques, ordonné depuis Python 3.7.
- `range(start, stop, step)` : suite entière générée à la demande, sans stockage en mémoire.

1.10 Conseils avant de continuer

1. Toujours se demander si l'on a besoin de **modifier** la collection : si non, un `tuple` est souvent préférable à une liste, ne serait-ce que pour éviter une modification accidentelle.
2. Ne jamais écrire `ma_liste = ma_liste.sort()` : la méthode `.sort()` renvoie `None`. Utiliser `sorted()` si l'on a besoin d'une nouvelle liste triée.

3. Préférer `.get()` à l'accès direct par crochets (`dico["cle"]`) dès qu'une clé pourrait être absente.
4. Utiliser un `set` dès que l'on doit éliminer des doublons ou tester une appartenance sur une grande collection.
5. Ne pas convertir un `range()` en liste « par réflexe » : s'il ne sert qu'à parcourir ou tester une appartenance, il est déjà optimal tel quel.
6. S'entraîner à représenter des données réseau réelles (parc d'équipements, table de routage, règles de pare-feu, inventaire) avec le type le plus adapté : c'est la meilleure façon de mémoriser durablement les différences entre ces cinq types.